Sequential Prefetching with a Stream Buffer: Implementation and Empirical Evaluation

Aengus McGuinness, Ali Saffrini, Madison Gates, Jacob Tom

Abstract

This report presents an implementation and empirical evaluation of Jouppi's stream buffer prefetcher [1] within a Pin-based cache hierarchy simulator. The stream buffer is modeled as a set of independent FIFO queues sitting between the L1 data cache and L2. On an L1D miss the simulator checks whether the missed block resides at the head of any active stream; a hit pops the head and prefetches the next tail line, keeping the buffer perpetually full. A miss allocates a new sequential stream beginning one block past the faulting address. The design is evaluated against three SPEC CPU2006 workloads—`libquantum`, `hmmer`, and `dealII`—across a sweep of buffer depths (0–8 lines), stream counts (1–16), and L1D associativities (1–8-way). Results confirm the theoretical prediction that the prefetcher is highly beneficial for purely sequential workloads, harmful in terms of L2 bandwidth for irregular workloads, and marginally effective for mixed-access workloads.

1 Implementation

1.1 Cache Hierarchy

The simulator models a three-level hierarchy: an L1 instruction cache, an L1 data cache, an L2 unified cache, and a memory object that counts main-memory accesses. Each level is derived from a common `cache` base class that implements LRU replacement, tag/set extraction, and hit/miss accounting. Cache parameters (block size, total size, associativity) are read from a configuration file at startup.

The L1D (`l1dcache`) overrides the standard `addressRequest` method to insert the stream buffer into the miss path. On every L1D miss the simulator first queries the stream buffer; only on a combined miss does it fall through to L2 and allocate a new stream.

1.2 Stream Buffer Data Structure

The stream buffer is implemented in the `stream_buf` class. Its state consists of an array of N stream descriptors, each holding a validity flag and a `head` field—the block-aligned address of the oldest prefetched line currently in that stream's FIFO:

```
struct Stream {
    bool          valid;
    unsigned long head;
};
```

The depth D is not stored explicitly per stream entry; instead, the invariant is that when a stream

is valid, blocks `[head, head + D*B)` have already been issued to L2 as prefetches, where B is the block size.

1.3 Access Protocol

When `l1dcache::addressRequest` records an L1D miss it calls `stream_buf::access(address)`. The method scans all N stream descriptors for a head match:

```
for( int i = 0; i < nStreams; i++ ) {
    if( !streams[i].valid || streams[i].head != blockAddr )
        continue;

    hits++;
    unsigned long newTailAddr =
        streams[i].head + (unsigned long)depth * blockSz;
    streams[i].head += blockSz;
    if( nextLevel != nullptr )
        nextLevel->addressRequest(newTailAddr);
    return true;
}
return false;
```

On a stream-buffer hit, the head advances by one block and a single new tail request is issued to L2. This keeps the buffer perpetually D lines deep without re-issuing the lines that are still valid ahead of the head.

1.4 Allocation Protocol

On a combined L1D and stream-buffer miss, `allocate_stream` is called. It selects a stream slot—preferring an unused (invalid) slot and falling back to round-robin eviction—and issues D consecutive prefetch requests to L2 starting one block past the faulting address:

```
unsigned long startAddr = blockAddr + blockSz;
for( int i = 0; i < depth; i++ )
    nextLevel->addressRequest(
        startAddr + (unsigned long)i * blockSz);
streams[idx].valid = true;
streams[idx].head  = startAddr;
```

The faulting line itself is fetched by the L1D miss handler; the stream buffer begins one block *ahead* of it, matching Jouppi's original design.

1.5 L1D Miss Path

The complete L1D miss path is:

1. Record the miss; identify the LRU eviction candidate.
2. Query `stream_buf::access(address)`.
3. If hit: the line is delivered; L1D is filled. No L2 demand fetch.
4. If miss: issue a demand fetch to L2; call `allocate_stream(address)`.

5. In both cases, install the new line at the LRU slot and update LRU metadata.

2 Experimental Methodology

All experiments use a fixed base configuration: a 64-byte block, 32 KB 8-way L1I, and a 512 KB 8-way L2. Experiments vary the L1D configuration. Simulations are capped at five billion retired instructions per run, executed under Intel Pin with 102 total configurations spread across four sweeps:

- **Associativity sweep.** L1D varies over 1, 2, 4, 8-way with no stream buffer. Isolates conflict-miss behavior from prefetch effects.
- **Depth sweep.** Direct-mapped L1D, one stream, depth $D \in \{0, 1, 2, 4, 8\}$.
- **Stream-count sweep.** Direct-mapped L1D, depth = 1, streams $N \in \{1, 2, 4, 8, 16\}$.
- **2D sweep.** Direct-mapped L1D, all combinations of $D \in \{0, 1, 2, 4, 8\}$ and $N \in \{1, 2, 4, 8\}$.

Three benchmarks are evaluated: `libquantum` (quantum circuit simulation, highly sequential array traversals), `hmmer` (hidden Markov model sequence matching, irregular pointer-chasing), and `dealII` (finite element analysis, a mix of sequential element iteration and sparse irregular access).

The two reported metrics are:

- **Effective L1D misses**: L1D misses minus stream-buffer hits. Represents accesses that actually stall waiting for L2.
- **Total L2 requests**: demand fetches plus all prefetch requests issued by the stream buffer. Represents L2 bus bandwidth consumed.

3 Results and Analysis

3.1 Associativity Sweep

Table 1: L1D misses vs. L1D associativity (no stream buffer).

Associativity	libquantum	hmmer	dealII
1-way	121.8M	1086.9M	485.0M
2-way	119.8M	657.2M	430.7M
4-way	119.6M	237.0M	418.8M
8-way	119.5M	156.9M	425.4M

`libquantum` is almost entirely insensitive to associativity (only a 1.9% reduction from 1-way to 8-way), confirming that its misses are compulsory or capacity in nature, not conflict. The working set simply exceeds any reasonable direct-mapped L1D, and the dominant access pattern is a linear scan that will always cold-miss the first time through.

`hmmer` drops 85.6% from 1-way to 8-way, a textbook conflict-miss signature. Its irregular, data-dependent accesses map many active lines to the same sets in a direct-mapped cache. Increasing associativity resolves this thrashing without any change to the access pattern itself.

`dealII` shows a moderate 13.7% improvement from 1 to 4-way, then flattens slightly at 8-way. This suggests a mix of conflict misses (resolved by modest associativity) and capacity/compulsory misses that associativity alone cannot address.

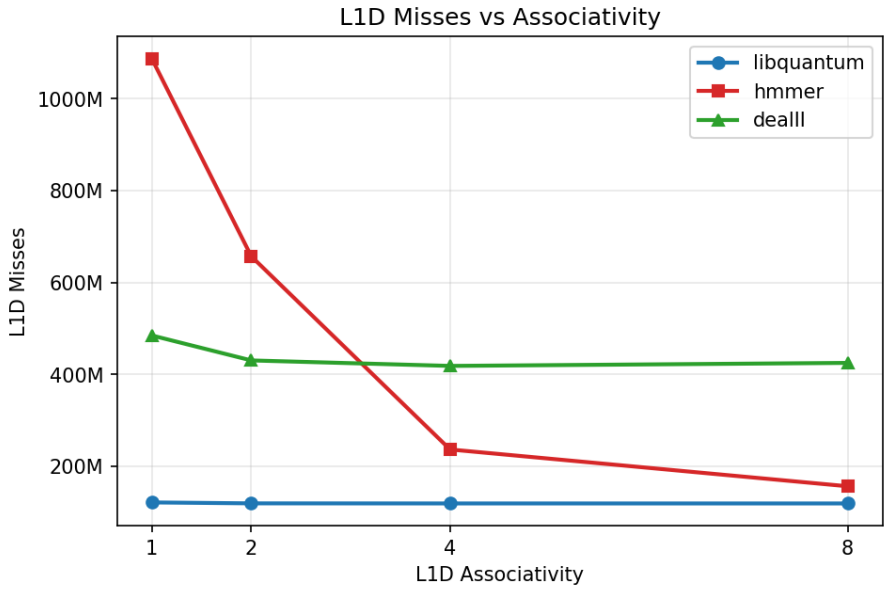


Figure 1: L1D misses vs. associativity for all three benchmarks (no stream buffer). `hmmer` is dominated by conflict misses; `libquantum` is insensitive to associativity.

3.2 Stream Buffer Depth Sweep

Table 2: Effective L1D misses and total L2 requests vs. buffer depth (direct-mapped L1D, 1 stream).

	Effective L1D Misses			Total L2 Requests		
Depth	libquantum	hmmer	dealII	libquantum	hmmer	dealII
0	121.2M	1089.3M	488.6M	121.2M	1091.1M	525.7M
1	2.8M	1061.8M	477.0M	124.0M	2130.3M	999.8M
2	2.8M	1061.1M	481.5M	126.8M	3189.9M	1489.9M
4	2.8M	1062.8M	477.0M	132.5M	5320.4M	2430.9M
8	2.8M	1062.6M	484.3M	143.7M	9570.2M	4404.3M

libquantum. The stream buffer reduces effective misses from 121.2M to 2.8M at depth = 1—a **97.7% reduction**. Increasing depth beyond 1 yields no further improvement: the effective miss count is essentially constant at 2.8M for $D \in \{1, 2, 4, 8\}$. The remaining 2.8M misses are compulsory cold misses on the first pass through each array that no amount of prefetching can eliminate.

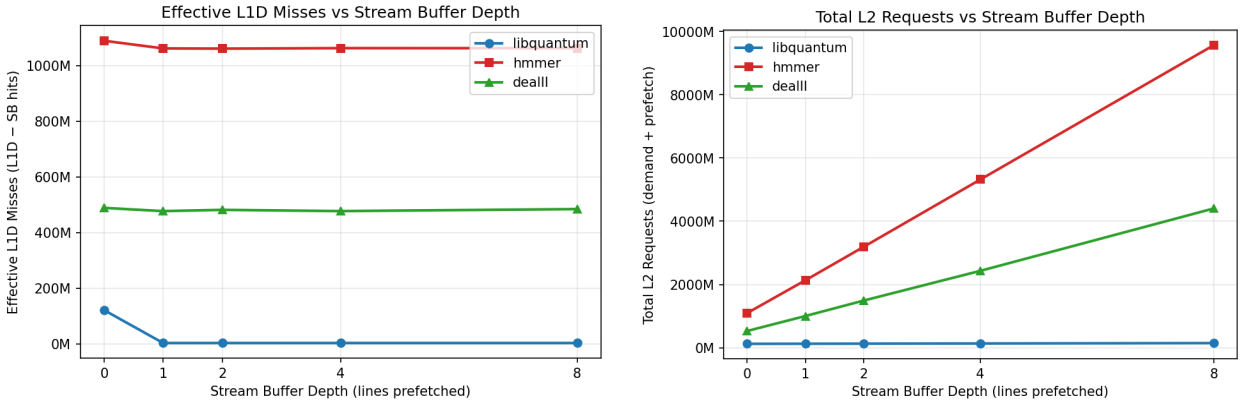
This saturation is a direct consequence of the simulator being *functional* rather than timing-based. The hit condition in `stream_buf::access` checks only whether the demand address matches `streams[i].head`; there is no model of prefetch latency or memory access time. A prefetched line is treated as immediately available the moment it is requested, regardless of depth. As a result,

depth has *no effect* on hit rate in this simulator—it only controls how many prefetch requests are issued to L2 per allocation and per hit. In real hardware, depth would matter significantly: a deeper buffer keeps more lines simultaneously in flight from L2, providing a longer runway to hide L2 latency between successive sequential accesses. A depth-1 buffer in real hardware would stall whenever the inter-access time is shorter than the L2 round-trip latency.

L2 bandwidth grows modestly with depth for `libquantum` (121M to 144M at $D = 8$, an 18.6% increase) because nearly every prefetched line is eventually consumed—the prefetch traffic replaces what would otherwise have been demand misses rather than adding net new traffic.

hmmer. Effective misses drop only 2.5% at any depth, confirming that the access pattern is fundamentally non-sequential. The stream buffer allocates a new stream on every L1D miss but the following access almost never falls at the predicted head address. Meanwhile, L2 traffic grows at approximately $(D+1) \times 1091$M requests—each of the $\sim$1089M per-miss stream allocations issues D useless prefetch requests to L2. At $D = 8$ this produces 9570M L2 requests, an **8.8**$\times$ increase over the no-buffer baseline, with essentially no benefit.

dealII. Effective misses fall by 2.4% at $D = 1$ and then plateau with slight noise. L2 requests grow roughly linearly with D, reaching 4404M at $D = 8$ (an 8.4$\times$ increase). The modest miss reduction reflects sequential iteration over finite-element mesh data, while the dominant cost—irregular sparse-matrix and pointer-structure accesses—is entirely untouched by the prefetcher.



(a) Effective L1D misses (L1D − SB hits).

(b) Total L2 requests (demand + prefetch).

Figure 2: Effect of stream buffer depth ($N = 1$ stream, direct-mapped L1D). `libquantum` sees a 97.7% miss reduction at $D = 1$ with negligible L2 overhead; `hmmer` gains almost nothing while L2 traffic grows 8.8$\times$.

3.3 Stream Count Sweep

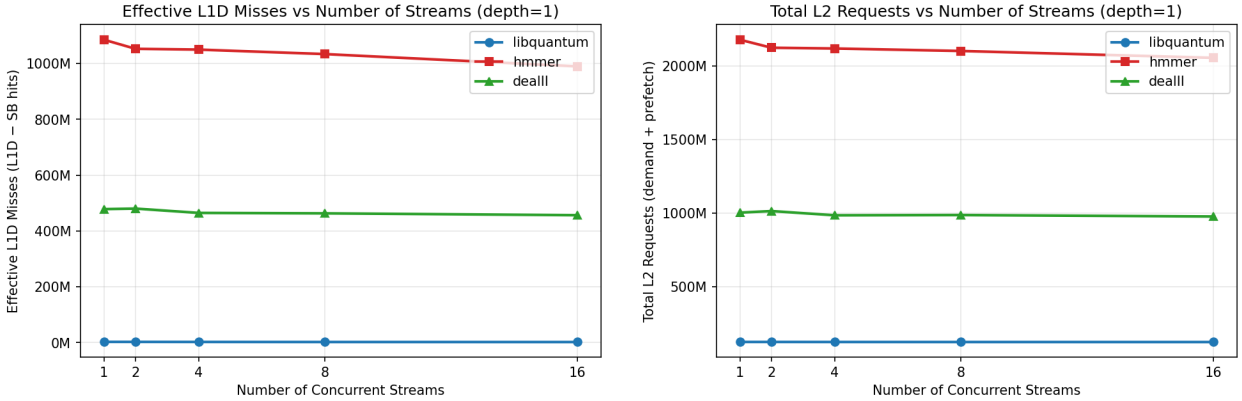
Table 3: Effective L1D misses vs. number of concurrent streams (depth = 1, direct-mapped L1D).

Streams	libquantum	hmmer	dealII
1	2.8M	1085.4M	478.5M
2	2.8M	1052.9M	480.4M
4	2.6M	1050.1M	464.9M
8	2.3M	1034.1M	463.4M
16	2.2M	990.0M	456.8M

For `libquantum`, a single stream is sufficient. The benchmark's sequential traversal is a single logical stream, so adding more slots is redundant; the marginal improvement from 1 to 16 streams is only 21%. L2 bandwidth is essentially flat (124.0M at $N = 1$ vs. 123.4M at $N = 16$).

For `hmmer`, more streams provides slow, modest gains. Going from 1 to 16 streams reduces effective misses by 8.8%. This suggests that a tiny fraction of the access pattern does exhibit short sequential runs across multiple concurrent streams (e.g., scoring multiple sequence positions simultaneously), but the gain is far from sufficient to justify the added hardware.

For `dealII`, multiple streams provide a 4.5% improvement from $N = 1$ to $N = 16$, consistent with the benchmark iterating over several data arrays concurrently (e.g., node coordinates, element stiffness matrices) where distinct streams can each track a separate array traversal. L2 bandwidth decreases slightly with N (from 1002.6M to 976.1M) because more streams capture more hits, reducing the number of new stream allocations and their associated prefetch floods.



(a) Effective L1D misses.

(b) Total L2 requests.

Figure 3: Effect of stream count ($D = 1$, direct-mapped L1D). Adding streams yields diminishing returns for `libquantum` (already saturated at $N = 1$) and modest improvements for `hmmer` and `dealII`.

4 Discussion and Conclusions

Are the results expected? Yes. The outcomes align precisely with theoretical predictions for Jouppi-style stream buffers.

The stream buffer is highly effective for libquantum. A 97.7% reduction in effective L1D misses at the minimum depth demonstrates that the workload's access pattern—long sequential scans over large arrays exceeding the L1D capacity—is exactly what stream buffers are designed to exploit.

The stream buffer is counter-productive for hmmer. Although effective misses drop by only 2.5%, L2 bandwidth explodes nearly 9× at $D = 8$. A real system would experience L2 and memory bus saturation, likely degrading performance despite the small miss-count improvement. The correct strategy for `hmmer` is increased associativity (an 85% miss reduction) rather than prefetching.

dealII occupies a middle ground. Its mixed sequential-irregular access pattern yields small but real prefetch benefits (∼6% at best) with proportional L2 bandwidth costs. The most cost-efficient operating point is $D = 1$, $N = 4$–8, which captures most of the benefit without the severe bandwidth waste seen at higher depths.

Design Implications. These results underscore two fundamental principles of hardware prefetching. First, *coverage* depends on matching the prefetch strategy to the access pattern: stream buffers excel at sequential access but are blind to irregular patterns. Second, *bandwidth* is a finite resource: prefetchers that generate many useless requests pollute the memory hierarchy even when they do not increase demand-miss latency directly.

References

[1] Norman P. Jouppi. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*, pages 364–373, 1990.